Advanced Java features Lab

This second lab has a set of short broken programs that you have to fix. Your TA will work through the first few with you; then you can continue in pairs. Finish what you can in the room and complete the rest on your own.

The eight programs

Each program lives in its own folder, `ch1` through `ch8`, as a single `.java` file, sometimes next to a data file it reads.

Several of these programs need more than one class, and here they all sit in the same file. Java allows that: only a `public` class has to live in a file of its own, so a file may hold one public class plus any number of non-public ones. It is convenient for this toy program, but usually it is a better idea to give each class its own file.

Each program carries its own description in a comment at the top of its main file, and sometimes a hint about what to look for.

Each program is only a few lines long. Read all of it before you run it.

- **ch1, ch4, ch5 and ch8 do not compile.** The compiler tells you where to look, even if it is not always clear about what you did wrong (it can't read your mind, after all).
- **ch2 compiles and then crashes.** Nothing is wrong until the program is already running.
- **ch3, ch6 and ch7 compile, run to completion, and give you a wrong answer without complaining.** Nothing at all tells you there is a problem.

That last group is the dangerous one, and those three are handled differently: their comment tells you what the program is supposed to *do*, but not what it should print. Working out what the right answer would have been is part of the job, because that is the position you are in with every program you write yourself.

For each folder:

1. Read the comment at the top of the file and decide what the program is supposed to do.
2. Compile it and run it, from inside its folder:

   ```
   cd ch1
   javac ShapeCatalog.java
   java ShapeCatalog
   ```

3. Work out why it does not do that.
4. **Write down the diagnosis in one sentence before you fix anything.** If you cannot say what is wrong, you are not ready to change the code. Guessing until the compiler stops complaining is not debugging!
5. Fix it.

When you need to know what a library method actually promises, the reference is the API documentation: https://docs.oracle.com/en/java/javase/21/docs/api/. Your TA will show you how to read it. Get used to going there rather than to a search engine or LLM.

Compiler error messages are written for people who already know the answer, which is a problem when you are learning. Read them anyway: the file and line number are almost always right, even when the wording is unhelpful, and the phrases in them are worth recognising the next time.

What is where

`ch1`	`ShapeCatalog`	does not compile
`ch2`	`GradeAverage`	crashes
`ch3`	`WordCount`	wrong answer
`ch4`	`WordBox`	does not compile
`ch5`	`AccountSummary`	does not compile
`ch6`	`PayReport`	wrong answer
`ch7`	`AnimalSounds`	wrong answer
`ch8`	`ReceiptPrinter`	does not compile

Open the file named in the second column to find out what each one is supposed to do.

Writing a small system

The eight programs above were about reading code. This one is about writing it.

`university/` contains a single file, `Driver.java`, which you may not modify. It uses six types that do not exist yet, and your job is to write them. Compile it now:

```
cd university
javac Driver.java
```

It fails, with one error per missing piece. **Read that error list as a to-do list** — this is the normal way to start from a specification in Java, and the compiler is better at tracking what is still missing than you are.

The types you have to write

- `Person` — abstract. Every person has a `name` and an `id`, supplied when they are created, and can report their `name` when asked. Every person can also `describe()` themselves in one line, but there is no sensible description of a person-in-general, so `Person` declares that method without writing it.
- `Student` — a `Person` who also has a `year`. Describes itself as `Emmy Noether (s01), undergraduate year 1`.
- `GradStudent` — a `Student` who is paid a fixed stipend of 2000 a month, and whose `advisor` is an `Instructor` — the object itself, not their name as a `String`. Describes itself as `Ada Lovelace (g01), graduate student in year 1 advised by Prof. Courant`, which means asking the advisor for its name rather than storing a copy of it.
- `Instructor` — a `Person` who has a monthly `salary` and is paid it. Describes itself as `Prof. Courant (i01), instructor`.
- `Paid` — the contract for anything the university pays: it can report its `monthlyPay()`. Notice which two classes need it, and where they sit relative to each other in the hierarchy.
- `Roster<T>` — holds people of one particular kind. It can `add` one, `get` the one at position `i`, and report its `size`.

Where `Roster` keeps its members

Use an `ArrayList<T>` and give it the three methods you need: `add(item)`, `get(i)` and `size()`.

Notice what `Roster` itself can and cannot do. It can hold members and hand them back, but it cannot call `describe()` on them: a plain `T` might be *any* type at all, and the compiler will not let you call a method it cannot prove exists. That is why `Driver` does the printing rather than `Roster`. We will discuss a solution to this in class.

`Driver` gets away with it because *there* the type is known. `students` is a `Roster<Student>`, so `students.get(i)` hands back a `Student` and not an `Object`, and no cast is needed. Compare that with `ch2`, where a raw `ArrayList` handed back an `Object` and the cast was a gamble. That is what the type parameter buys you.

Something to think about while you write

`Driver` puts a `GradStudent` into a `Roster<Student>`, and puts a `GradStudent` and an `Instructor` into the same `Paid []`. Both work, for two different reasons. Make sure you can say what they are.

Expected output

```
-- students --
Emmy Noether (s01), undergraduate year 1
Alan Turing (s02), undergraduate year 2
Srinivasa Ramanujan (s03), undergraduate year 1
Ada Lovelace (g01), graduate student in year 1 advised by Prof. Courant
-- staff --
Prof. Courant (i01), instructor
monthly payroll: 11000.0
```